

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-212БВ-24

Студент: Колосов Е. Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 03.10.25

Москва, 2025

Постановка задачи

Вариант 5.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы

Общий метод и алгоритм решения

Использованные системные вызовы:

1. pipe(int pipefd[2])

- Назначение: создаёт канал (pipe), по которому родительский и дочерний процессы могут обмениваться данными.

2. fork(void)

- Назначение: создаёт дочерний процесс, который является копией родительского.

3. dup2(int oldfd, int newfd)

- Назначение: дублирует файловый дескриптор oldfd в newfd.

4. execl(const char *path, const char *arg0, ..., (char *)NULL)

- Назначение: заменяет образ текущего процесса на новый исполняемый файл.

5. read(int fd, void *buf, size_t count)

- Назначение: читает данные из файлового дескриптора.

6. write(int fd, const void *buf, size_t count)

- Назначение: записывает данные в файловый дескриптор.

7. open(const char *pathname, int flags, mode_t mode)

- Назначение: открывает файл (или создаёт, если его нет).

8. close(int fd)

- Назначение: закрывает файловый дескриптор.

9. wait(pid_t pid)

- Назначение: ожидает завершения дочернего процесса.

10. exit(int status)

- Назначение: завершает процесс с указанным статусом.

В рамках лабораторной работы была реализована программа, состоящая из двух процессов — родительского и дочернего, взаимодействующих через канал. Программа написана на языке C с использованием только системных вызовов POSIX, без библиотек `stdio.h`, `threads.h` и `stdatomic.h`. Родительский процесс считывает числа из `stdin` и передаёт их дочернему процессу через `pipe`. Дочерний процесс проверяет числа на простоту: составные числа записываются в файл, а при получении простого или отрицательного числа оба процесса завершаются. Взаимодействие между процессами обеспечивается с помощью системных вызовов `pipe`, `fork`, `dup2`, `execl`, `read`, `write`, `open`, `close`, `wait` и `exit`. Программа корректно обрабатывает числа и завершает работу в соответствии с условиями задания.

Код программы

parent.c

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int IsPrime(int number) {
    if (number <= 1) {
        return 0;
    }
    if (number == 2) {
        return 1;
    }
    if (number % 2 == 0) {
        return 0;
    }
    for (int index = 3; index * index <= number; index += 2) {
        if (number % index == 0) {
            return 0;
        }
    }
    return 1;
}

int main() {
    int pipe1[2];
    if (pipe(pipe1) == -1) {
        const char error_message[] = "pipe1 failed\n";
        write(STDERR_FILENO, error_message, sizeof(error_message) - 1);
        exit(1);
    }
```

```

char fileName[256];
const char prompt[] = "Enter filename: ";
write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

int index = 0;
char character;
while (read(STDIN_FILENO, &character, 1) == 1 && character != '\n') {
    fileName[index++] = character;
}
fileName[index] = '\0';

pid_t processId = fork();
if (processId == 0) {
    // дочерний процесс
    close(pipe1[1]);
    dup2(pipe1[0], STDIN_FILENO);
    close(pipe1[0]);

    execl("./child", "child", fileName, NULL);
    const char exec_error[] = "exec failed\n";
    write(STDERR_FILENO, exec_error, sizeof(exec_error) - 1);
    exit(1);
} else if (processId > 0) {
    // родительский процесс
    close(pipe1[0]);

    int number = 0;
    int sign = 1;
    char inputChar;

    while (read(STDIN_FILENO, &inputChar, 1) == 1) {
        if (inputChar == '-') {
            sign = -1;
        } else if (inputChar >= '0' && inputChar <= '9') {
            number = number * 10 + (inputChar - '0');
        } else if (inputChar == '\n') {
            number *= sign;

            char debug_msg[32];
            int len = 0;
            int temp = number;
            if (temp == 0) debug_msg[len++] = '0';
            else {
                if (temp < 0) { debug_msg[len++] = '-'; temp = -temp; }
                int start = len;
                while (temp > 0) { debug_msg[len++] = (temp % 10) + '0'; temp /= 10; }
                int end = len - 1;
                while (start < end) { char t = debug_msg[start]; debug_msg[start] = debug_msg[end];

```

```

debug_msg[end] = t; start++; end--; }
    }
    debug_msg[len++] = '\n';
    write(STDERR_FILENO, "Sending: ", 9);
    write(STDERR_FILENO, debug_msg, len);

    write(pipe1[1], &number, sizeof(number));
    if (number < 0 || IsPrime(number)) {
        break;
    }
    number = 0;
    sign = 1;
}
}

close(pipe1[1]);
wait(NULL);
} else {
    const char fork_error[] = "fork failed\n";
    write(STDERR_FILENO, fork_error, sizeof(fork_error) - 1);
    exit(1);
}

return 0;
}

```

child.c

```

#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int IsPrime(int number) {
    if (number <= 1) {
        return 0;
    }
    if (number == 2) {
        return 1;
    }
    if (number % 2 == 0) {
        return 0;
    }
    for (int index = 3; index * index <= number; index += 2) {
        if (number % index == 0) {
            return 0;
        }
    }
}

```

```

    return 1;
}

int main(int argumentCount, char* argumentValues[]) {
    if (argumentCount != 2) {
        const char usage_message[] = "Usage: child <output_file>\n";
        write(STDERR_FILENO, usage_message, sizeof(usage_message) - 1);
        exit(1);
    }

    int fileDescriptor = open(argumentValues[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fileDescriptor == -1) {
        const char open_error[] = "open failed\n";
        write(STDERR_FILENO, open_error, sizeof(open_error) - 1);
        exit(1);
    }

    int number;
    while (read(STDIN_FILENO, &number, sizeof(number)) == sizeof(number)) {
        if (number < 0 || IsPrime(number)) {
            close(fileDescriptor);
            exit(0);
        }

        char buffer[32];
        int temp = number;
        int index = 0;

        if (temp == 0) {
            buffer[index++] = '0';
        } else {
            if (temp < 0) {
                buffer[index++] = '-';
                temp = -temp;
            }
            int start = index;
            while (temp > 0) {
                buffer[index++] = (temp % 10) + '0';
                temp /= 10;
            }
            // Reverse digits
            int end = index - 1;
            while (start < end) {
                char temp_char = buffer[start];
                buffer[start] = buffer[end];
                buffer[end] = temp_char;
                start++;
                end--;
            }
        }
    }
}

```

```

    }
}

buffer[index++] = '\n';
write(fileDescriptor, buffer, index);
}

close(fileDescriptor);
return 0;
}

```

Протокол работы программы

```

heavenfiend@Rust-PC2:~/ipc_lab$ ./parent
Enter filename: output123.txt
20
Sending: 20
22
Sending: 22
15
Sending: 15
16
Sending: 16
3
Sending: 3
heavenfiend@Rust-PC2:~/ipc_lab$ cat output123.txt
20
22
15
16
heavenfiend@Rust-PC2:~/ipc_lab$ |

```

Вывод

В ходе лабораторной работы была реализована программа, обеспечивающая взаимодействие между родительским и дочерним процессами через канал. Программа корректно обрабатывает числа, записывает составные в файл и завершает работу при получении простого или отрицательного числа. Все требования задания выполнены, использованы только системные вызовы POSIX. Работа показала, как можно организовать межпроцессное взаимодействие в операционных системах на базе Unix.